

IS216 Lab Test 1

[25 marks]

General Instructions:

- This is a time-bound (1 hour 30 minutes), open-book, closed-Internet, no-LLM, and **individual** test. You may visit **ONLY** the white-listed websites (within ExamenA) during the test.
- You must test your web pages using **Google Chrome Web Browser** (Version 139.x or later). Your graders will be using only **Google Chrome Web Browser** (Version 139.x) to test your web pages.
- **No questions will be entertained** by the IS216 teaching team or the invigilators during the test period. If necessary, make your own assumptions and proceed to complete test questions.
- You must **use only standard HTML5, CSS, Bootstrap (Version 5.3), Vue and Axios** in your solutions unless the question specifies otherwise. Do not use any other third-party libraries (e.g. Angular, React, or others).
- Unnecessary use of innerHTML is prohibited. Any unnecessary use of innerHTML found in your code will result in a penalty of 1 mark per occurrence.
- Use meaningful names for HTML class/id and JavaScript variables and functions. You must indent your code (HTML/CSS/JavaScript) properly. Failure to do so will attract a penalty of up to **20%** of your score for the corresponding question.
- You **MUST** include your name as author in the comments of all your submitted source files. Failure to do so will attract a penalty of up to **20%** of your score for the corresponding question. For example, if your registered name is "KIM Jong Un" and email ID is kim.jongun.2024, include the following comment at the beginning of each source file you write.

HTML Files	CSS & JavaScript Files
<pre><!-- Name: KIM Jong Un Email: kim.jongun.2024 --></pre>	<pre>/* Name: KIM Jong Un Email: kim.jongun.2024 */</pre>

- You may wish to comment out the parts in your code which cause errors. Commented code will not be marked.

Academic Integrity

- All student submissions will be thoroughly checked by an SMU-approved source code plagiarism checker software program AND an additional external software program. The source code checking will be conducted across all submissions from all sections of IS216.
- Suspected plagiarism cases will be reported immediately to the IS216 faculty in charge and SCIS Dean's Office for further investigation.
- Students in the suspected cases will be informed accordingly by their section faculty, and the incident will be escalated to the SMU University Council of Student Conduct.

Submission Instructions

- **Due Date**

- **3 October 2025 (Friday) 5:30 PM Singapore Time**
 - This deadline **may vary** for students with time extensions (*please check with your invigilator for details*).
- Late submission policy is as follows:

Submit within 5 minutes of set deadline	10% penalty of your entire test's score
Beyond 5 minutes	0 mark

- **Zip** up all files in **Q1/Q2/Q3/Q4** folders into **<YOUR_SMU_ID>.zip**
 - For example, **kim.jongun.2024.zip**
 - **Verify by unzipping this zip file – check the content inside**
 - **Incorrect submission file name** WILL attract a penalty of up to **20%** of your score for the entire test.
- Only **zip** format is accepted.
 - **.7z, rar** or other **compression formats** are **NOT** accepted.
 - Until the correct **zip** format is submitted again by the student, it will be assumed that the student has NOT made the submission and late submission policy will apply.
- Submit the **zip** file in **Examina**:
 - **It is your (student's) individual responsibility to ensure that the zip file submission was successful.**
 - **Your section faculty, instructor, and Teaching Assistants will NOT verify the submission for you.**

Legend



Do NOT edit this given resource file.



Your answer/code goes here into this given resource file.

Q1. Basic CSS

[6 Marks]

Given resources in folder client/src/components/q1

-  q1.vue
-  q1.css
-  q1_readonly.css

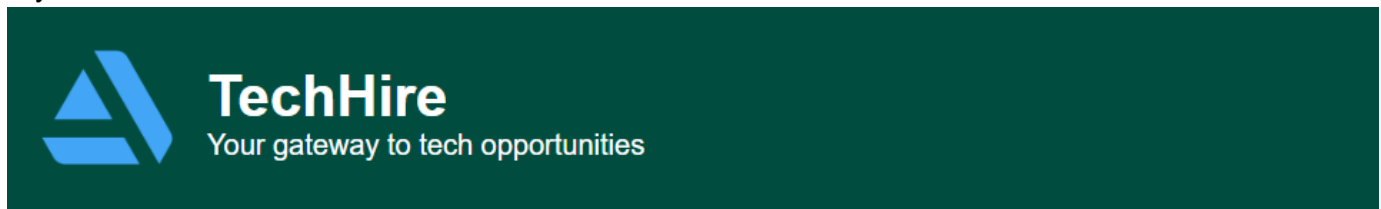
Instructions

1. You are to style the HTML elements using **vanilla CSS (no Bootstrap)**.
2. Open *q1.vue* in a browser and examine how it looks.
3. You are to edit only *q1.css* file.
4. You are NOT to add new selectors in *q1.css* file except where stated to do so.

Note: You are to define CSS styles in the labelled locations e.g. /* part A */ within the CSS file to match the screenshots to the best of your interpretation **where the requirement is not stated explicitly**. This includes margin, padding, font style, font size, font weight, etc.

Part A (2 marks)

Style the header as shown below.



- Set the width of the logo image to 100 pixels.
- Set the spacing surrounding the text to be 20 pixels all round.
- Align the logo image and text vertically at the middle of the header.
- Set the color of the text as white or #fff in hexadecimal.
- Set the background color of the header as #004d40 in hexadecimal.

Part B: Style the divs for “Filter Jobs” and “Open Positions” as shown below. (1.5 mark)

Filter Jobs

Role:

Location:

Role	Company	Location	Type
Frontend Developer	 PixelCraft Award-winning web agency	Remote	Full-time
Database Analyst	InfoMedia	Remote	Full-time
Data Analyst Intern	DataDive	On-site	Internship
Backend Engineer	CodeNest	Remote	Contract

- Align “Filter Jobs” and “Open Positions” divs vertically and towards the top.
- Set the radius of the border to 8 pixels.
- For “Filter Jobs” div, set the spacing on the right of its right border to be 20 pixels.
- For “Open Positions” div, set the spacing on the left of its left border to be 20 pixels.

Part C (1 mark)

Style the “Filter Jobs” form as shown below.

Current: Role: Location:

After styling: Role: Location:

30% 60%

- The width of the form labels should be 30% and the width for the form controls should be 60% of the form width.

Part D (0.5 marks)

Style the table as shown below by adding a new selector.

Role	Company	Location	Type
Frontend Developer	New! PixelCraft Award-winning web agency	Remote	Full-time
Database Analyst	InfoMedia	Remote	Full-time
Data Analyst Intern	DataDive	On-site	Internship
Backend Engineer	CodeNest	Remote	Contract

- The even rows of the table would have background color #f1f1f1 in hexadecimal.

Part E (1 mark)

Style the first cell on the first row of the “Open Positions” table as shown below. (1 mark)

Frontend Developer New!

- Position the “New!” badge on the top right corner of the cell.

The final page should look like this.


TechHire
 Your gateway to tech opportunities

Filter Jobs
 Role:
 Location:

Open Positions

Role	Company	Location	Type
Frontend Developer	New! PixelCraft Award-winning web agency	Remote	Full-time
Database Analyst	InfoMedia	Remote	Full-time
Data Analyst Intern	DataDive	On-site	Internship
Backend Engineer	CodeNest	Remote	Contract

© 2025 TechHire. All rights reserved.

Q2. Bootstrap: Genshin Character Gallery

[5 Marks]

Given resources in folder `client/src/components/q2`

 `Q2.vue`

Instructions

1. You will **edit** `Q2.vue`.
2. You are to solve the web page layout problem by using **Bootstrap 5.3** grid, utilities, breakpoints, and built-in JavaScript (tabs) only.
3. Do not add custom CSS or external libraries.
4. Do not change IDs or provided text.
5. You are not required to add/edit other styles, such as margin, padding, font style, font size, etc., not mentioned in the requirements.
6. Test at small, medium, and large screen widths.

Part A (2 marks)

When the browser width is ≥ 992 px (**1g** breakpoint), q2.html must display:

- The header is displayed at the top.
- Below the header, the layout is a **3-column grid**:
 1. **Left column** (spans 3 grid columns)
 - The 4 character names must:
 - a. **stack vertically** and
 - b. **be left-aligned**
 2. **Middle column** (spans 5 grid columns)
 - Character note form
 3. **Right column** (spans 4 grid columns)
 - Selected character image
- The footer is displayed at the bottom.

q2.html
(screen width 992px)

Genshin Character Gallery

Choose a character to view art and a short blurb.

Albedo

Sayu

Sucrose

Thoma

Character Notes

Click a name to switch the featured artwork. Use the form to log a quick note about the selected character.

Your nickname

Region

Traveler

Select

Why this character

Short reason

Save note



© 2025 WAD2 Corporation

Part B (2 marks)

When the browser width is ≥ 768 px (**md** breakpoint) but < 992 px (**lg** breakpoint), q2.html must display:

- The header is displayed at the top.
- Two rows:
 - Row 1: `#contentCol` (spans 6 grid columns) and `#imageCol` (spans 6 grid columns) side by side
 - Row 2: `#navCol` spans **full width** below Row 1 and is **horizontally centered**
- Achieve the reading order using **Bootstrap order utilities**, not HTML reordering.
- The footer is displayed at the bottom.

q2.html
(screen width 768px)

Genshin Character Gallery

Choose a character to view art and a short blurb.

Character Notes

Click a name to switch the featured artwork.
Use the form to log a quick note about the selected character.

Your nickname

Region

Select
▼

Why this character

Short reason

Save note



Albedo

Sayu

Sucrose

Thoma

© 2025 WAD2 Corporation

Part C (1 mark)

When the browser width is < 768 px (md breakpoint), q2.html must display:

- The header is displayed at the top.
- Single column in this order: header → #navCol → #imageCol → #contentCol.
- Image stays full width.
- The footer is displayed at the bottom.

q2.html (screen width 767px)

Genshin Character Gallery
 Choose a character to view art and a short blurb.

Albedo
Sayu
Sucrose
Thoma



Character Notes
 Click a name to switch the featured artwork. Use the form to log a quick note about the selected character.

Your nickname

Region

Select
▼

Why this character

Short reason

Save note

© 2025 WAD2 Corporation

Q3. Basic Vue

[9 Marks]

Given resources in folder Q3

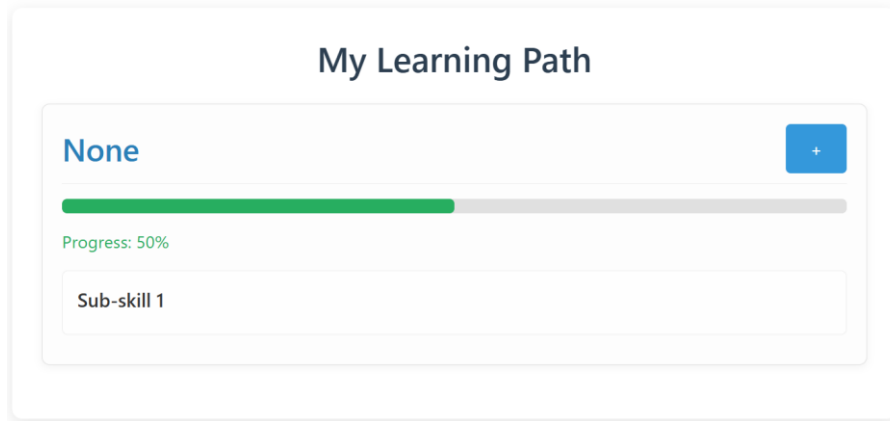
 Q3.vue

Instructions

1. You are to edit Q3.vue.
2. In Q3.vue, the “learningPath” array has been populated with the list of main skills and sub-skills.
3. Q3.vue will be marked by an automated test script which may overwrite “learningPath” array with new content.

Original Output

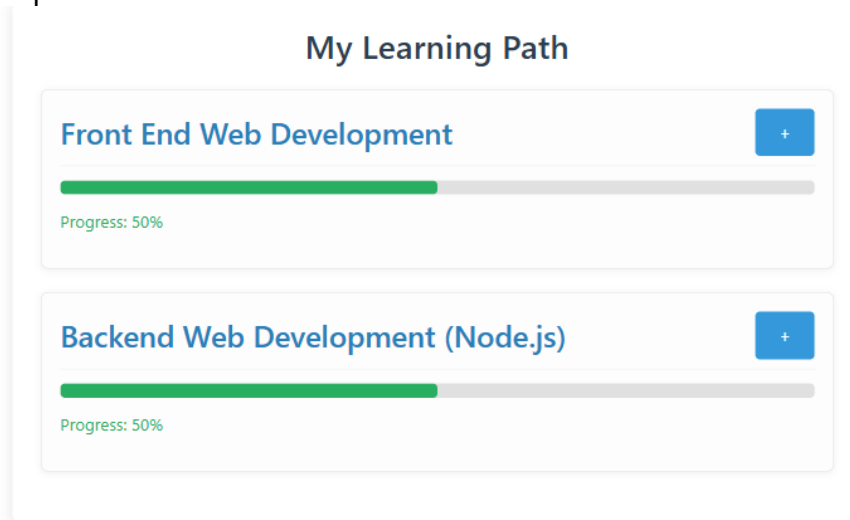
When Q3.vue is loaded on the browser for the first time, it will show the following output.



Part A (2 marks)

Load Main Skills

1. Fill in codes to show the list of main skills as retrieved from the “learningPath” array.
2. The list of main skills should be displayed when the web page is loaded.
3. Note that the list of main skills displayed on the web page would change if the contents of “learningPath” array is updated.
4. The desired output is shown below.



Part B (2 mark)

Load Sub Skills

1. Fill in codes to display the list of sub-skills for the specific main skill.
2. The desired output is shown below.

My Learning Path

Front End Web Development

Progress: 50%

- HTML Fundamentals
- CSS Basics (Layout & Styling)
- JavaScript Fundamentals (Variables, Functions, Loops)
- DOM Manipulation & Event Handling
- Asynchronous JavaScript (Fetch API)
- Introduction to React.js

Backend Web Development (Node.js)

Progress: 50%

- Node.js Basics
- Express.js Fundamentals

Part C (3 marks)

Toggle Display of Sub-Skills

1. Fill in codes to display the list of sub-skills for the specific main skill when the toggle button beside the main skill is clicked.
2. You will have to add codes to handle the click event of the button.
3. Each click of the button should change its label i.e. the button label should toggle from “+” to “-” and then back.
4. Clicking on “+” should display the list of sub-skills and clicking on “-” should remove the list of sub-skills from display.

5. The desired output is shown below.

After clicking
“+” on first
main skill

The screenshot shows a web application titled "My Learning Path". It features two main skill cards. The first card, "Front End Web Development", is highlighted with a blue background and a blue button with a minus sign (-) in the top right corner. Below the title is a green progress bar at 50% and a list of topics: HTML Fundamentals, CSS Basics (Layout & Styling), JavaScript Fundamentals (Variables, Functions, Loops), DOM Manipulation & Event Handling, Asynchronous JavaScript (Fetch API), and Introduction to React.js. The second card, "Backend Web Development (Node.js)", has a white background and a blue button with a plus sign (+) in the top right corner. It also shows a green progress bar at 50%.

After clicking
“-” on first
main skill

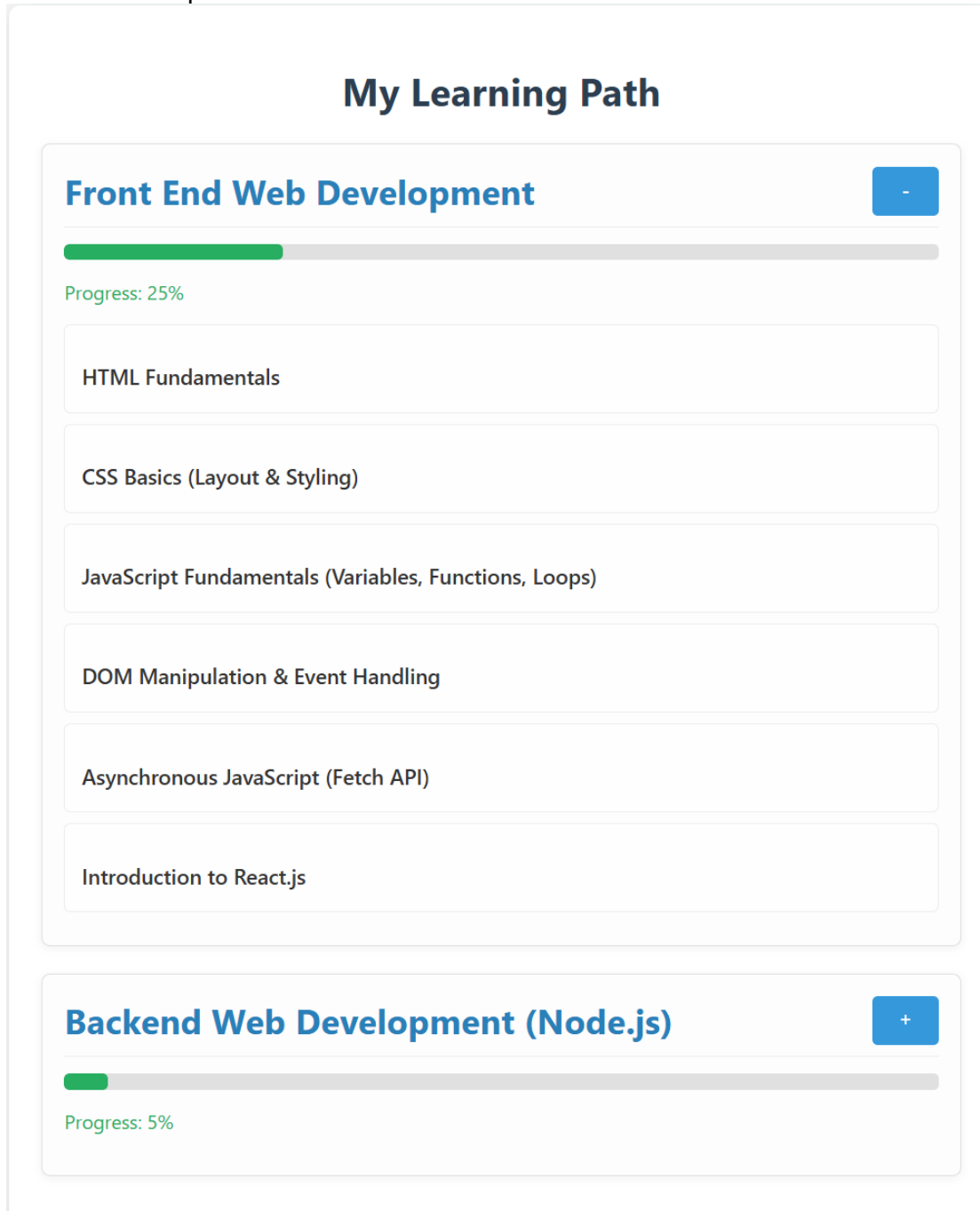
The screenshot shows the same "My Learning Path" interface. In this state, the "Front End Web Development" card has a white background and a blue button with a plus sign (+) in the top right corner. The "Backend Web Development (Node.js)" card remains unchanged with a white background and a blue button with a plus sign (+). Both cards show a green progress bar at 50%.

Part D (2 marks)**Show Completion Progress of Main Skill**

1. Fill in codes to show the completion progress of each main skill as a progress bar.
2. The completion progress of each main skill is to be computed by averaging the completion progress across all sub-skills listed under that specific main skill, rounded to the closest integer value e.g. 25.2 would be rounded to 25 and 25.5 would be rounded to 26.
3. Create a computed property `mainSkillsProgress` which returns an object whose keys are main skills' IDs and values are the completion progress.

```
/*
Part D: Add computed property mainSkillsProgress for progress bar
{ "101": 25 , "102": 5 }
*/
const mainSkillsProgress = null;
```

4. The desired output is shown below.



Q4: Vue - AXIOS & JSON**[5 Marks]**Given resources in folder **src/components/q4**

Q4.vue

Instructions

1. **Edit only** Q4.vue.
2. Use **Axios** for all HTTP calls.
3. You will have to start the backend server.js using “node server.js” in the “server” folder.
4. No external libraries except Axios are allowed. Do not add custom CSS.
5. Do **NOT** hardcode. Grading will be done using another **requests.json** file containing different data.

Scenario & APIStudents submit **study room booking requests**.Each request has the shape: { id, name, slot, status } where status is "pending", "approved", or "denied". Here is the **initial** requests.json file.**requests.json**

```
[
  {
    "id": 1,
    "name": "Alex Tan",
    "slot": "2025-12-25 10:00",
    "status": "approved"
  },
  {
    "id": 2,
    "name": "Meera Lim",
    "slot": "2025-12-25 11:00",
    "status": "pending"
  },
  {
    "id": 3,
    "name": "Kai Ong",
    "slot": "2025-12-25 13:00",
    "status": "pending"
  },
  {
    "id": 4,
    "name": "Jon Koh",
    "slot": "2025-12-25 14:00",
    "status": "denied"
  }
]
```

API Documentation

- Please see in **Pages 20-22** examples of GET and POST requests using **Postman**.

Part A (0.5 marks)

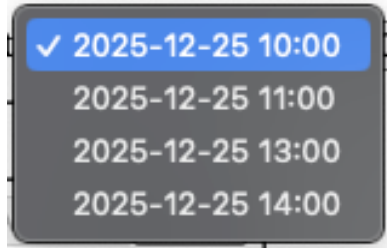
Implement `loadData()` to Axios **GET** `http://localhost:8000/api`, storing the relevant data in `slots` and `requests` reactive variables.

- Use an appropriate debugger (e.g. console logging both global variables and checking in the Developer Tools → Console) to verify that `loadData()` works as expected.

Part B (0.5 marks)

Insert codes to display all **slots** as dropdown menu options inside `<select id="slotSelect"></select>`.

- Do **NOT** *hardcode*.

Click on this dropdown menu	All slots read from requests.json must be shown:
Select a slot: 2025-12-25 10:00	

Part C (1 mark)

Insert codes to display all **requests** inside `<tbody id="tbodyRequests">`.

- Each request must be a **table row** consisting of the following **table columns**:
 - Name
 - Slot (date/time string)
 - Status
 - Actions
 - Two buttons: **Approve** and **Deny**.
 - If a request's status is **approved**:
 - the **Approve** button must be *disabled* (greyed out)
 - the **Deny** button must be *enabled*
 - If a request's status is **denied**:
 - the **Approve** button must be *enabled*
 - the **Deny** button must be *disabled* (greyed out)
 - If a request's status is **pending**:
 - the **Approve** button must be *enabled*
 - the **Deny** button must be *enabled*
- Do **NOT** *hardcode*.

Assume that **requests.json** is in its initial state as shown on **Page 13** where 1 request is “approved”, 2 requests are “pending” and 1 request is “denied”. Loading **Q4.vue** in web browser will display as follows:

q4.html (fresh page load with the original requests.json)

Study Room Requests

Your name: Select a slot:

Name	Slot	Status	Actions
Alex Tan	2025-12-25 10:00	approved	Approve Deny
Meera Lim	2025-12-25 11:00	pending	Approve Deny
Kai Ong	2025-12-25 13:00	pending	Approve Deny
Jon Koh	2025-12-25 14:00	denied	Approve Deny

Part D (1.5 marks)

Implement **updateStatus()** to facilitate the following:

- Clicking **Approve** or **Deny** sends Axios **POST** with `action=update` with id and status.
- On success, the HTML table must show the latest status ('approved' or 'denied') of the updated request.
- The **Approve** button and the **Deny** button must be **correctly disabled or enabled** as per the latest status.

Assume that **requests.json** is in its initial state as shown on **Page 13** where 1 request is “approved”, 2 requests are “pending” and 1 request is “denied”.

Next, the user clicks on the **Approve** button for **Meera Lim**'s request.

- See how the **Approve** button is now **disabled (greyed out)**.

q4.html (after)

Study Room Requests

Your name: Select a slot:

Name	Slot	Status	Actions
Alex Tan	2025-12-25 10:00	approved	Approve Deny
Meera Lim	2025-12-25 11:00	approved	Approve Deny
Kai Ong	2025-12-25 13:00	pending	Approve Deny
Jon Koh	2025-12-25 14:00	denied	Approve Deny

And this **change of status** for **Meera Lim**'s request is reflected in **requests.json** (the API will have updated the JSON file):

requests.json

```
[
  {
    "id": 1,
```

```

    "name": "Alex Tan",
    "slot": "2025-12-25 10:00",
    "status": "approved"
  },
  {
    "id": 2,
    "name": "Meera Lim",
    "slot": "2025-12-25 11:00",
    "status": "approved"
  },
  {
    "id": 3,
    "name": "Kai Ong",
    "slot": "2025-12-25 13:00",
    "status": "pending"
  },
  {
    "id": 4,
    "name": "Jon Koh",
    "slot": "2025-12-25 14:00",
    "status": "denied"
  }
]

```

Next, the user clicks on the **Deny** button for **Kai Ong**'s request.

- See how the **Deny** button is now **disabled (greyed out)**.
- Similarly, this action will result in **requests.json** file being updated accordingly.

q4.html (after)

Study Room Requests

Your name: Select a slot:

Name	Slot	Status	Actions
Alex Tan	2025-12-25 10:00	approved	Approve Deny
Meera Lim	2025-12-25 11:00	approved	Approve Deny
Kai Ong	2025-12-25 13:00	denied	Approve Deny
Jon Koh	2025-12-25 14:00	denied	Approve Deny

Next, the user clicks on the **Approve** button for **Jon Koh's** request.

- See how the **Approve** button is now **disabled (greyed out)**.
- Similarly, this action will result in **requests.json** file being updated accordingly.

q4.html (after)

Study Room Requests

Your name: Select a slot: 2025-12-25 10:00 ▾ Create

Name	Slot	Status	Actions
Alex Tan	2025-12-25 10:00	approved	Approve Deny
Meera Lim	2025-12-25 11:00	approved	Approve Deny
Kai Ong	2025-12-25 13:00	denied	Approve Deny
Jon Koh	2025-12-25 14:00	approved	Approve Deny

Part E (1.5 marks)

When the user fills out the “**Your name**” form field, selects a **date/time slot** in the dropdown menu, and clicks on the **Create** button:

- Retrieve 1) “**Your name**” form field input and 2) user-selected “**slot**” dropdown menu option
- No form field validation is required.
- Axios **POST** with `action=create` with form data `{ name, slot }`.
- On success:
 - Add the new request to the top of the HTML table.
 - Clear the form fields:
 - Empty the “**Your name**” form field
 - Set the “**Slot**” dropdown menu option to the default one

For example, the user has filled out the form as shown below and clicks on the **Create** button:

- **Name:** Taylor Swift
- **Slot:** 2025-12-25 10:00

q4.html (fresh page load with the original requests.json)

Study Room Requests

Your name: Select a slot: 2025-12-25 10:00 ▾ Create

Name	Slot	Status	Actions
Alex Tan	2025-12-25 10:00	approved	Approve Deny
Meera Lim	2025-12-25 11:00	pending	Approve Deny
Kai Ong	2025-12-25 13:00	pending	Approve Deny
Jon Koh	2025-12-25 14:00	denied	Approve Deny

See in the next page for the updated q4.html...

The new request by “Taylor Swift” is displayed **at the top of the HTML table** with the status of “pending”.

q4.html (after “Taylor Swift” clicks on the Create button)

Study Room Requests

Your name: Select a slot: 2025-12-25 10:00 ▼

Name	Slot	Status	Actions
Taylor Swift	2025-12-25 10:00	pending	Approve Deny
Alex Tan	2025-12-25 10:00	approved	Approve Deny
Meera Lim	2025-12-25 11:00	pending	Approve Deny
Kai Ong	2025-12-25 13:00	pending	Approve Deny
Jon Koh	2025-12-25 14:00	denied	Approve Deny

Next, another user “Eunwoo Cha” fills out the form and clicks on the Create button:

q4.html

Study Room Requests

Your name: Select a slot: 2025-12-25 10:00 ▼

Name	Slot	Status	Actions
Taylor Swift	2025-12-25 10:00	pending	Approve Deny
Alex Tan	2025-12-25 10:00	approved	Approve Deny
Meera Lim	2025-12-25 11:00	pending	Approve Deny
Kai Ong	2025-12-25 13:00	pending	Approve Deny
Jon Koh	2025-12-25 14:00	denied	Approve Deny

The new request by “Eunwoo Cha” is displayed **at the top of the HTML table** with the status of “pending”.

q4.html

Study Room Requests

Your name: Select a slot: 2025-12-25 10:00 ▼

Name	Slot	Status	Actions
Eunwoo Cha	2025-12-25 10:00	pending	Approve Deny
Taylor Swift	2025-12-25 10:00	pending	Approve Deny
Alex Tan	2025-12-25 10:00	approved	Approve Deny
Meera Lim	2025-12-25 11:00	pending	Approve Deny
Kai Ong	2025-12-25 13:00	pending	Approve Deny
Jon Koh	2025-12-25 14:00	denied	Approve Deny

And the two new requests should be reflected in **requests.json** (the API will have updated the JSON file):

requests.json

```
[
  {
    "id": 6,
    "name": "Eunwoo Cha",
    "slot": "2025-12-25 10:00",
    "status": "pending"
  },
  {
    "id": 5,
    "name": "Taylor Swift",
    "slot": "2025-12-25 10:00",
    "status": "pending"
  },
  {
    "id": 1,
    "name": "Alex Tan",
    "slot": "2025-12-25 10:00",
    "status": "approved"
  },
  {
    "id": 2,
    "name": "Meera Lim",
    "slot": "2025-12-25 11:00",
    "status": "pending"
  },
  {
    "id": 3,
    "name": "Kai Ong",
    "slot": "2025-12-25 13:00",
    "status": "pending"
  },
  {
    "id": 4,
    "name": "Jon Koh",
    "slot": "2025-12-25 14:00",
    "status": "denied"
  }
]
```

Example of **api.php** GET call using **Postman**

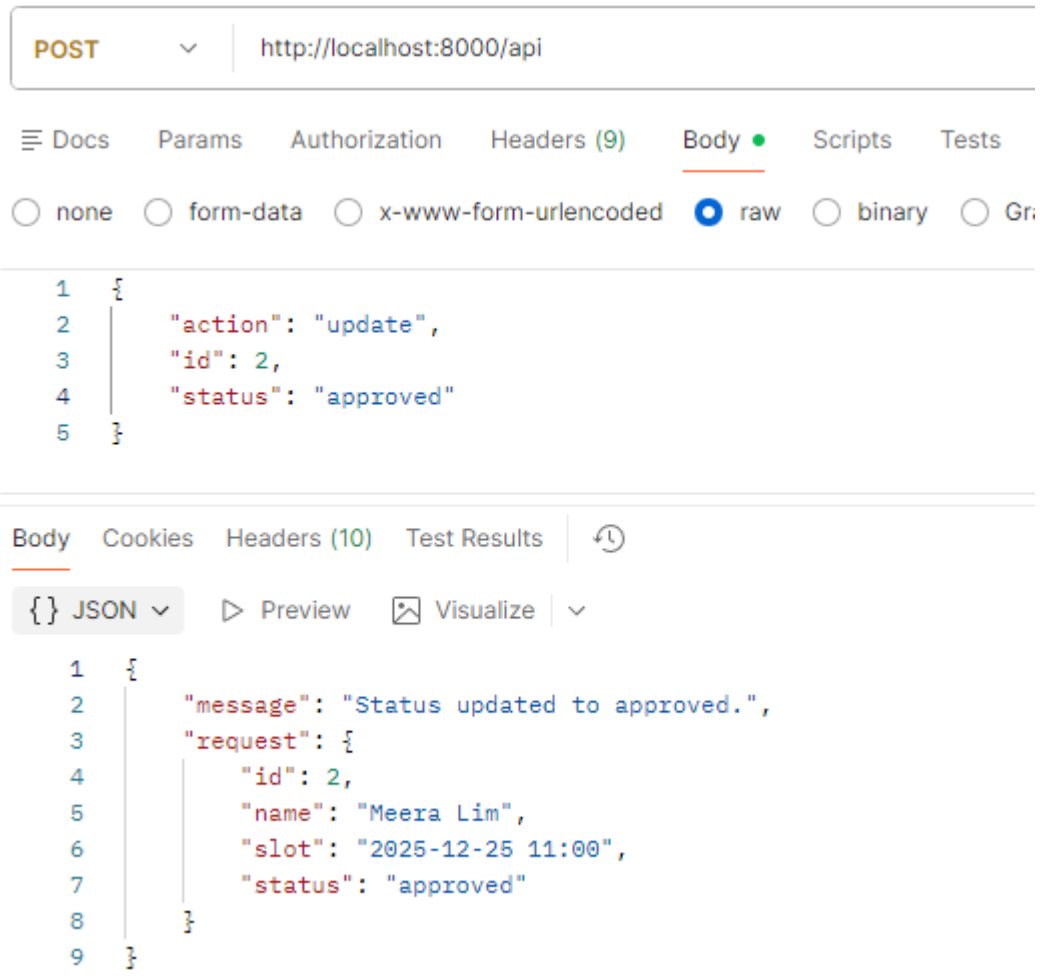
The screenshot shows the Postman interface for a GET request to `http://localhost:8000/api`. The response is displayed in JSON format, showing a list of slots and requests.

```

1  {
2    "slots": [
3      "2025-12-25 10:00",
4      "2025-12-25 11:00",
5      "2025-12-25 13:00",
6      "2025-12-25 14:00"
7    ],
8    "requests": [
9      {
10       "id": 1,
11       "name": "Alex Tan",
12       "slot": "2025-12-25 10:00",
13       "status": "approved"
14     },
15     {
16       "id": 2,
17       "name": "Meera Lim",
18       "slot": "2025-12-25 11:00",
19       "status": "denied"
20     },
21     {
22       "id": 3,
23       "name": "Kai Ong",
24       "slot": "2025-12-25 13:00",
25       "status": "approved"
26     },
27     {
28       "id": 4,
29       "name": "Jon Koh",
30       "slot": "2025-12-25 14:00",
31       "status": "denied"
32     }
33   ]
34 }

```

Example of **api.php** POST call using **Postman**

Updating the status to “approved” for the request with id value of 2	 The screenshot shows the Postman interface for a POST request to <code>http://localhost:8000/api</code>. The request body is a JSON object: <code>{ "action": "update", "id": 2, "status": "approved" }</code>. The response body is a JSON object: <code>{ "message": "Status updated to approved.", "request": { "id": 2, "name": "Meera Lim", "slot": "2025-12-25 11:00", "status": "approved" } }</code>.
Updating the status to “denied” for the request with id value of 3	

The screenshot shows the Postman interface for a POST request. The URL bar at the top shows `http://localhost:8000/api`. The request body is set to 'raw' (JSON) and contains the following JSON:

```

1  {
2    "action": "update",
3    "id": 3,
4    "status": "denied"
5  }

```

Below the request body, the 'Body' tab is selected, showing the response in JSON format. The response is:

```

1  {
2    "message": "Status updated to denied.",
3    "request": {
4      "id": 3,
5      "name": "Kai Ong",
6      "slot": "2025-12-25 13:00",
7      "status": "denied"
8    }
9  }

```

Example of **api.php** POST call using **Postman**

Creating a
new
request

The screenshot displays a REST client interface with the following components:

- Request Method and URL:** A dropdown menu shows "POST" and a text field contains "http://localhost:8000/api".
- Request Body:** The "Body" tab is selected. It shows a JSON object:

```
1 {
2   "action": "create",
3   "name": "Taylor Swift",
4   "slot": "2025-12-25 10:00"
5 }
```
- Response Body:** The "Body" tab is also selected for the response. It shows a JSON object:

```
1 {
2   "message": "Request created for Taylor Swift.",
3   "request": {
4     "id": 5,
5     "name": "Taylor Swift",
6     "slot": "2025-12-25 10:00",
7     "status": "pending"
8   }
9 }
```

End of Paper